Project: CNN network
Advanced Computer Architectures

()

Michiel Vandromme & Joffrey Van Cauwenberghe

May 11, 2024



Class: Master Software Engineering

1 Introduction

For the project of Advanced Computer Architectures, we have chosen to implement a CNN with CUDA. A Convolutional Neural Network (CNN) is a specific type of neural network primarily used for image classification and object detection. Through its various layers, it can extract features from input data and use them to recognize patterns. A CNN network is composed of a convolutional layer, a subsampling layer, and a fully connected layer.

In the convolutional layer, filters are applied to the input data to extract features. Each filter slides over the input and computes the convolution with the overlapping region of the input. This process results in the generation of feature maps.

These feature maps are then fed to the subsampling or pooling layer. The purpose of this layer is to reduce the dimensions of the feature maps. This is done to prevent overfitting and reduce computational complexity. The pooling operations are performed on non-overlapping regions of the feature map. Each pooling operation converts the values of such a region into a single value.

Finally, these reduced feature maps are used by the fully connected layer to generate probabilities for, for example, each class in a classification task.

CNNs often deal with very large datasets that need to be processed, which is why we are investigating how the GPU can be used to make the training process of a CNN more efficient. CNNs use various operations, such as convolution and matrix multiplications. These are all operations that are highly parallelizable and thus ideal to perform on the GPU.

The dataset utilized in our project comprises images of grayscale clothing items, sourced from the Zalando MNIST Fashion dataset. This dataset serves as an alternative to the traditional MNIST dataset, commonly used for digit recognition tasks. Figure 1 shows some examples.

2 Implementation

Before a CNN network can make predictions on the test data, it must first be trained on the training data to recognize patterns in the input data. During training, the parameters of the network, namely the weights and biases, are iteratively optimized to minimize the network's error. For each input, training consists of a 'forward pass' and a 'backward pass'.

During the 'forward pass', the input is fed into the network, and it passes through each layer of the network until an output is generated at the last layer. Based on this output and the expected output, the error is calculated. Then, this error is propagated back through the network to compute its gradients. The gradients indicate the impact of each weight on the error. Finally, based on these gradients, the weights and biases of each layer are adjusted. This process continues until the error falls below a certain threshold.

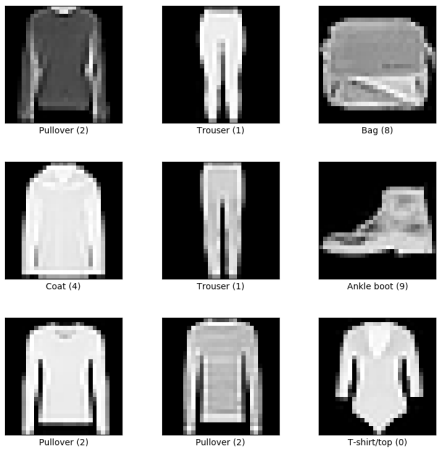


Figure 1: Examples of MNIST Zalando Fashion dataset

2.1 Forward pass

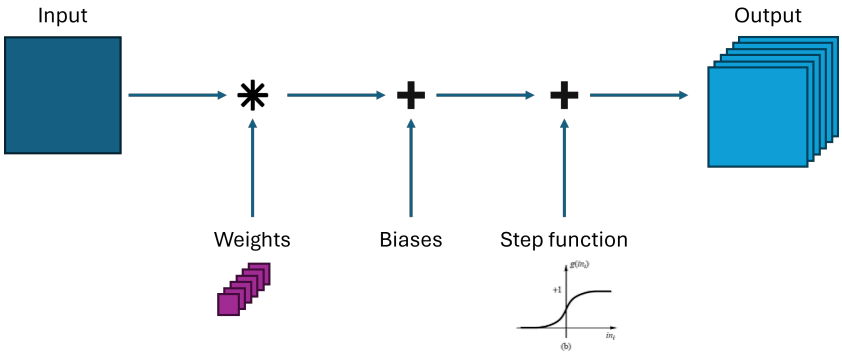


Figure 2: Operations in a layer

Although each layer serves a different function, the internal operations follow a similar concept. Figure 2 illustrates the sequence of operations. The first phase involves a linear transformation where a weighted sum with an additional bias is computed. In the final step, an activation function is used to introduce non-linearity into the network. This activation function can be a sigmoid, ReLU, tanh, etc. We did some small test runs with ReLu and Sigmoid, the two best known, and saw better results with the sigmoid. That is why we chose this activation function.

In the preactivation kernel of the convolutional layer, there is always a two-dimensional weight matrix per filter. Each filter slides over the input and computes the convolution value between the filter and the overlapping input region. All the computed convolution

values for a filter form a feature map. Since the input images are 28x28 and we chose six filters, each 5x5 in size, this results in six feature maps of size 24x24. Due to the relatively small size of our images, we chose to use only one block with 24x24 threads so that each thread calculates one preactivation value per feature map.

A very similar process occurs when calculating the preactivation values in the subsample layer. There, a pooling window, in our case of size 4x4, slides over the generated feature maps of the previous layer in a non-overlapping manner. It is basically the same as the convolution layer but with a stride that is as big as the dimension of the pooling window. Here we use one block of 6x6x6 threads. Each thread is responsible for one pooling window and thus calculates one value in one of the reduced feature maps.

The preactivation kernel for the fully connected layer converts the six feature maps into 10 output probabilities. For each of the 10 output neurons, a linear combination of the 6 input features is made. Each thread is responsible for the multiplication of one weight with one feature map for each output neuron. This results block of 6x6x6 threads.

An overview of the entire forward pass can be seen in Figure 3 by following the green lines.

2.2 Backward pass

In the backward pass, the computed error is propagated back through the layers of the network. Based on this error, gradients/derivates of the output, preactivation values, and weights of each layer can be calculated. These gradients are necessary because they contain information on how to adjust the biases and weights of each layer to minimize the error.

The calculation of the error at the end of the forward propagation can be seen in listing 1. This approach positively influences correct predictions and negatively impacts incorrect ones. By following the red lines in Figure 3, all backpropagation phases can be tracked.

```
1     IF class_i == label:
2         error[class_i] = 1 - output[class_i]
3     ELSE
4         error[class_i] = 0 - output[class_i]
5
```

Listing 1: Propagated error

Updating the derived weights, preactivation values, biases, and output is very similar in each layer. For instance, biases are updated in the convolutional layer by working with six blocks of 24x24 threads. Each of these threads takes 10% of a value from the derived preactivation, normalizes it with the total size of all values ($= 6^3$), and adds it to the respective bias.

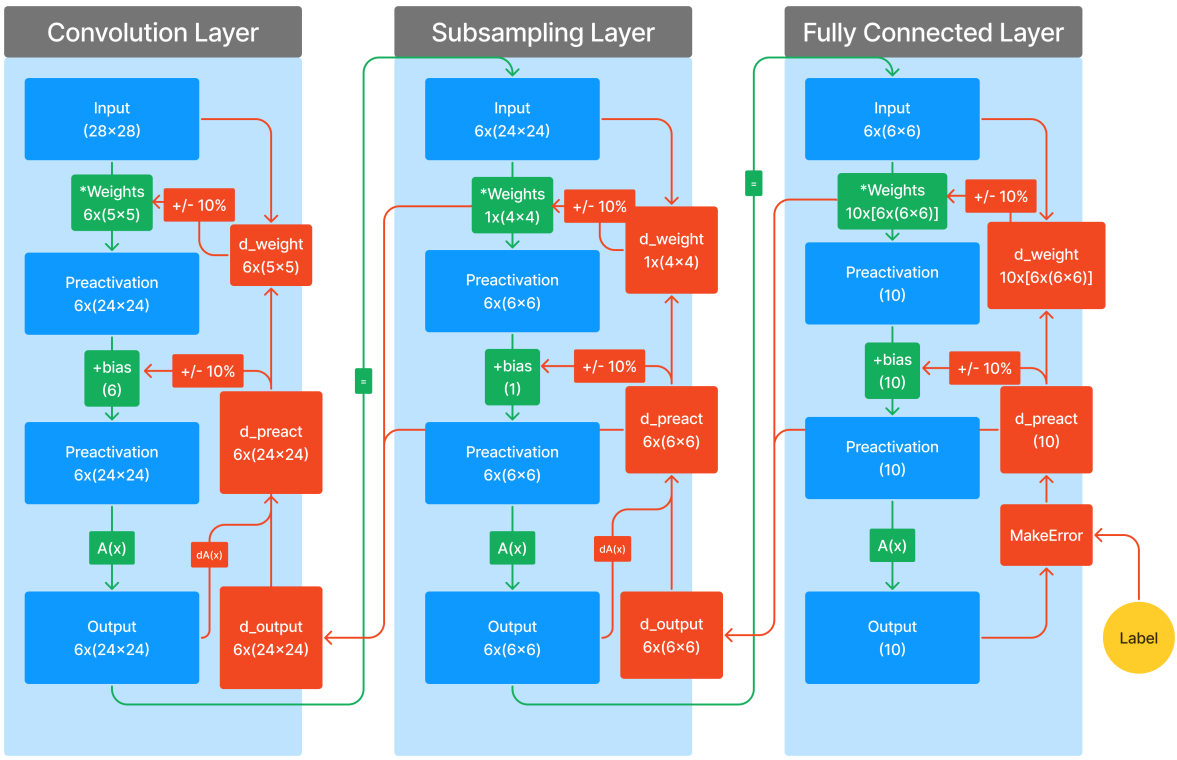


Figure 3: Oveview total CNN

Calculating the derived weights involves convolving the output of the previous layer with the derived preactivation values. The derived output of a layer, in turn, depends on the weights of the next layer and the derived preactivation values of that next layer.

Finally, the derived preactivation values are calculated by convolving the derived output values with the actual values of the output passed through the derivative of the activation function.

3 Evaluation & Conclusion

In a CNN, numerous parameters can influence the final error rate. These include factors such as the size and number of filters, the amount of training iterations to be completed, and the choice of activation function. As this is our first CNN network, we based the dimensions and flow of the network on this GitHub repository [1]. To initialise our initial weights and biases, we let them vary between -0.5 and +0.5. This way, we hope to leave

them fairly neutral at the beginning of training and let the backpropagation push them in the right direction.

With our implementation, utilizing the sigmoid activation function (see Figure) and 10 training iterations, we achieved an average error rate of 24.44%, but with some outliers towards 90%. This bad error rate is rare and must be due to some initial weight matrix that still causes 10 iterations to be insufficient to get the weights right. The results can be found in figure 4. Further research could explore the impact of the activation function and other parameters.

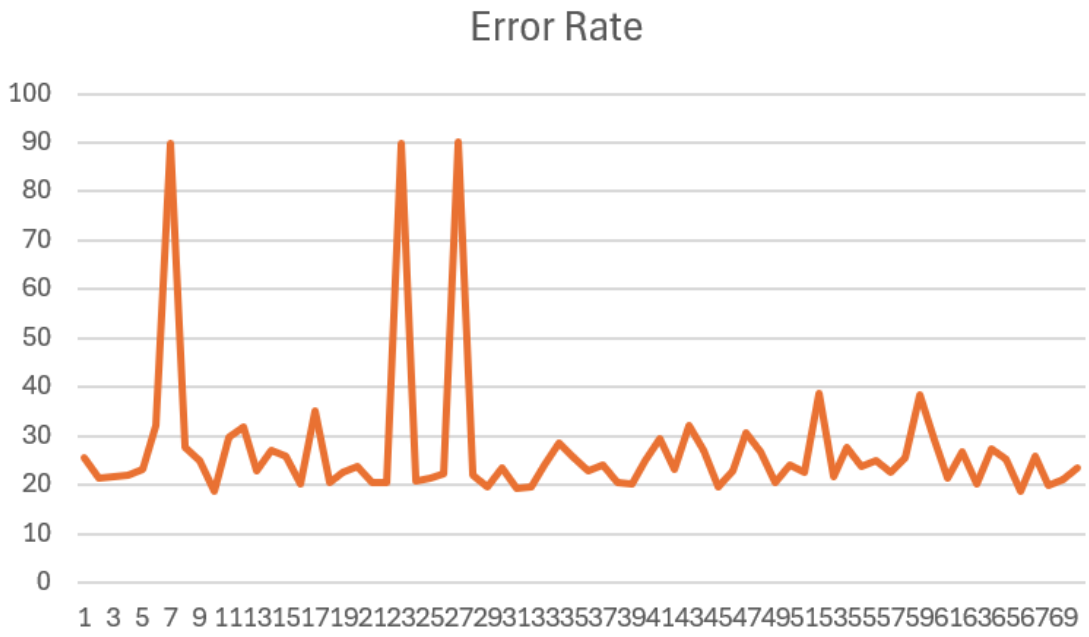


Figure 4: Error rate during 70 tests

Almost every operation performed in a CNN network is parallelizable. This allows us to write a kernel for almost every method and leverage the GPU extensively.

In our implementation, we made an effort to select the most efficient organization of threads (threadsPerBlock and number of blocks) for each kernel. This was achieved by considering the number of operations each thread needs to perform and assessing the memory requirements of each thread. As a result, we arrived at a more elegant implementation than the code we found online, by taking into account the dimensions of the matrices and filters more effectively. In the future, we could implement different thread organizations and investigate which one is most efficient.

You can find our code on this github repo: [2].

References

[1] P. Hanji, *Paramhanji/CUDA-CNN*, original-date: 2017-05-01T16:42:30Z, Apr. 22, 2024. [Online]. Available: `https://github.com/paramhanji/CUDA-CNN` (visited on 05/10/2024).

[2] M. Vandromme and J. V. Cauwenberghe, *Michiel2002v/cuda$_c$nn$_p$roject*. [Online]. Available: `https://github.com/michiel2002v/CUDA_CNN_PROJECT` (visited on 05/10/2024).

[3] A. Umer. "Understanding convolutional neural networks ÿ§ : A beginnerâ€™s journey into the architecture ÿš€," CodeX. (Aug. 9, 2023), [Online]. Available: `https://medium.com/codex/understanding-convolutional-neural-networks-a-beginners-journey-into-the-architecture-aab30dface10` (visited on 05/10/2024).